

Astronomía Práctica II: Introducción a IRAF

Lauren Flor Torres, PhD

1. Descripción general de IRAF

IRAF (*Image Reduction and Analysis Facility*) es un paquete de uso común en astronomía. Se utiliza para desarrollar procedimientos de análisis y reducción de datos astronómicos, tanto imágenes como espectros.

Su estructura se organiza en paquetes, dentro de los cuales existen tareas (*tasks*) o programas que realizan procedimientos específicos.

En el documento se indica que IRAF no es la única herramienta disponible, ni necesariamente la mejor o la peor; sin embargo, debido a su tiempo de desarrollo —desde 1984—, se ha convertido en una herramienta conocida y cómoda de usar.

Mediante actualizaciones se han incorporado paquetes nuevos, incluso asociados a instrumentos y/o telescopios. Por ejemplo, se mencionan STDAS, paquete de análisis del Space Telescope Science Institute, y paquetes del observatorio GEMINI.

Además, se destaca que IRAF sigue siendo software libre.

2. Componentes básicos del sistema

El sistema IRAF se describe a partir de varias piezas básicas. En el material se mencionan las siguientes:

- Lenguaje de comandos, CL: es la interfaz del usuario con el sistema.
- Paquetes de aplicación: contienen algoritmos para analizar datos reales.
- Sistema operativo virtual, VOS: es la interfaz del sistema huésped.

El lenguaje de comandos CL tiene como función básica proporcionar una interfaz entre el usuario y todos los paquetes de aplicación de IRAF. Muchos de los programas incluidos en los paquetes son funciones estándar para análisis de datos, y la mayoría son iterativos. La ejecución se controla con parámetros que el usuario puede modificar.

3. Paquetes y tareas

Un paquete es un conjunto de tareas o comandos que están lógicamente conectados. Para cargar o tener acceso a un paquete únicamente se necesita teclear su nombre y presionar Enter.

Por ejemplo, para cargar o tener acceso a las tareas del paquete `plot`, se teclea esa palabra. En el entorno de IRAF, esto puede escribirse como:

```
cl> plot
```

Al hacer esto, se mostrarán las tareas del paquete.

4. Representación de archivos de imágenes

Un archivo de imagen es una matriz de números que representa tripletas de la forma (x, y, I) , donde (x, y) son las coordenadas espaciales de un punto sobre la imagen e I es la intensidad.

En el caso de imágenes digitales, las imágenes están representadas por un número fijo finito de elementos de imagen, o píxeles. Los píxeles definen una discretización en un sistema de coordenadas cartesiano que referencia un punto sobre la imagen.

Cada píxel está caracterizado por sus coordenadas y su intensidad. Cuando se realizan operaciones sobre la imagen, realmente se opera sobre cada uno de los píxeles de la misma.

5. Formato FITS

FITS (*Flexible Image Transport System*) es el formato de archivo más utilizado comúnmente en astronomía.

Aunque el formato FITS es conocido por su uso en almacenamiento de imágenes, también se utiliza a menudo para almacenar datos que no son imágenes, como espectros, listas de fotones, cubos de datos, tablas y muchos otros tipos de datos.

Siendo un formato de almacenamiento binario, un archivo FITS puede contener varios bloques de datos, y cada uno de ellos podría contener distintos datos o imágenes de un objeto. Por ejemplo, es posible almacenar en un mismo archivo FITS imágenes en varias bandas de un mismo objeto.

Una de las mayores ventajas del formato FITS para su uso en ciencia es que, asociado a cada archivo o bloque de datos en el archivo, hay una cabecera con información legible en formato ASCII. Esto permite que un usuario pueda examinar las cabeceras de un archivo para investigar su procedencia y contenidos, determinar su origen y obtener demás información característica útil.

Esta particularidad resulta bastante útil al momento de manipular grandes cantidades de datos, administrar archivos en bases de datos, transportar datos y hacerlos duraderos y usables en el tiempo.

Cada archivo FITS consiste en una o varias cabeceras y su respectivo bloque de datos en formato binario. Cada cabecera contiene secuencias de 80 cadenas de caracteres fijos que llevan pares de valores en la forma de un *keyword* y su valor asociado.

Cada par de valores provee información importante en forma de metadatos, como el tamaño, origen, formato binario de los datos, comentarios, historia de los datos y cualquier otra información que el creador desee. Aunque se cuenta con algunas palabras restringidas para la cabecera de los archivos en formato FITS, el estándar permite el uso arbitrario de cualquier palabra.

6. Configuración inicial: archivo `login.cl`

El archivo `login.cl` contiene todos los aspectos específicos asociados al funcionamiento de la sesión de IRAF del usuario. Puede resultar conveniente editar este archivo para conseguir un comportamiento más cómodo del programa.

Por ejemplo, se puede editar mediante:

```
(iraf27) lauren@lauren-VirtualBox:~/iraf$ gedit login.cl
(iraf27) lauren@lauren-VirtualBox:~/iraf$ vi login.cl
```

El documento sugiere editar, por ejemplo, los siguientes parámetros:

```
set editor= emacs
set stdimage= imt8192
set imtype="fits"
```

7. Inicio de una sesión de IRAF

Para abrir la terminal `xgterm`, se pueden usar comandos como:

```
(iraf27) lauren@lauren-VirtualBox:~/iraf$ xgterm&
(iraf27) lauren@lauren-VirtualBox:~/iraf$ xgterm-sbr&
```

Dentro de la terminal abierta, se escribe nuevamente la activación del entorno:

```
(base) lauren@lauren-VirtualBox:~/iraf$ source activate iraf27
```

Luego se puede abrir el visualizador `ds9`:

```
(iraf27)lauren@lauren-VirtualBox:~/iraf$ ds9&
```

Finalmente, se inicia IRAF escribiendo:

```
(iraf27)lauren@lauren-VirtualBox:~/iraf$ cl
```

8. Consola de IRAF y cierre de sesión

Al ejecutar `cl`, se debe leer detenidamente el mensaje de bienvenida de la consola de IRAF. Un usuario familiarizado con paquetes como `gnuplot` notará cierta similitud con la consola de IRAF.

IRAF atiende instrucciones ingresadas por la línea de comandos del prompt y se ejecuta directamente sobre la consola `xgterm`. Cerrar la consola `xgterm` implicará cerrar la aplicación de IRAF.

Para cerrar una sesión IRAF basta con ejecutar el comando:

```
cl>logout
```

9. Paquetes cargados y navegación entre paquetes

Al ejecutar `cl`, automáticamente se cargan varios paquetes. El material indica que existe una forma de consultar qué paquetes ya están cargados; en el texto extraído no se conserva la línea exacta de ese comando.

Es importante tener en cuenta que CL distingue entre letras mayúsculas y minúsculas para el nombre de un comando o los argumentos.

Para volver al prompt básico de IRAF, `cl>`, se puede usar:

```
cl>bye
```

El comando `bye` descarga el paquete que se encuentre cargado en su instancia más inmediata. Se puede navegar a través de los paquetes hacia adelante usando el nombre del paquete, y hacia atrás con el comando `bye`.

10. Comandos generales de ayuda

10.1. Lista de tareas y paquetes

Para ver qué paquetes y tareas hay disponibles en el nivel actual, se usa el comando:

```
cl>?
```

Si se quiere ver qué hay contenido dentro de cada uno de los paquetes instalados en IRAF, sin tener que cargarlos, se puede ejecutar:

```
cl>??
```

Con `?nombre-del_paquete` se listan las tareas del paquete especificado, siempre que haya sido cargado previamente. Por ejemplo, de forma genérica:

```
?nombre-del_paquete
```

Los nombres de los paquetes y tareas pueden darse de forma abreviada, usando los primeros tres o cuatro caracteres, evitando ambigüedad. Si la abreviatura no es única, se tendrá un mensaje de error.

10.2. Ayuda sobre una tarea específica

Para saber en qué paquete se encuentra una tarea, el comando `help` proporciona esta información. Por ejemplo, si se desea saber dónde está la tarea `precess`, se puede teclear:

```
plot> help precess
```

En la parte superior central de la ayuda aparece la ruta de la tarea. En el ejemplo del documento, para la tarea `precess`, la ruta será `noao.astutil`.

Para ejecutar la tarea `precess`, se deberá cargar el paquete correspondiente. El documento indica que, si `noao` ya se encuentra cargado, se debe cargar `astutil`; de no ser así, se tendría que cargar antes el paquete `noao`.

10.3. Búsqueda de un término en la documentación: refer

Otro comando de ayuda particularmente poderoso es `refer`. El comando `refer` busca en la documentación de todos los paquetes una palabra en particular.

Por ejemplo, si se quiere ver cuáles paquetes hacen alusión al término `fft`, se ejecuta:

```
cl>refer fft
```

10.4. Recuperación de comandos anteriores: history

Si se quiere recuperar un comando que se ha ejecutado anteriormente, se pueden usar las flechas del teclado hacia arriba o hacia abajo, como en el shell del sistema operativo Linux.

Otra forma es usar el comando `history`. Este comando recupera la lista de últimos comandos ejecutados. Si se ejecuta sin argumentos, `history` recuperará los últimos 10 comandos, que es el número por defecto.

Si se quieren recuperar los últimos 23 comandos ejecutados en el shell de IRAF, se puede ejecutar:

```
cl>history 23
```

10.5. Lista de parámetros de una tarea: `lpar`

Para mostrar la lista de parámetros necesarios para el funcionamiento de una tarea se utiliza la tarea `lpar`. Por ejemplo:

```
cl>lpar imheader
```

Esto lista los parámetros de funcionamiento del task `imheader`.

10.6. Uso de `imheader`

El task `imheader` lee el header de un archivo FITS y lo imprime en pantalla. `lpar` muestra que hay varios parámetros que se pueden pasar al task `imheader` para su ejecución.

Ejecutando el task con valores por defecto:

```
cl>imheader dev$pix
```

Con los valores por defecto, esta es la información que se provee, y claramente no es toda la información contenida en la cabecera de la imagen.

Si se modifica uno de los parámetros del task, específicamente el parámetro `longheader=yes`, se obtendrá mucha más información. Así, la ejecución para ver la cabecera completa de la imagen sería:

```
cl>imheader dev$pix longheader=yes
```

11. Visualización de imágenes

Para visualizar una imagen hay que usar una herramienta auxiliar, `ds9`. El programa `ds9` es un paquete creado por el Smithsonian Astrophysical Observatory. Aunque es una herramienta que funciona independientemente de IRAF, se puede usar en conjunto con este como herramienta de visualización de imágenes.

Para abrir `ds9` desde IRAF se puede usar:

```
cl>!ds9&
```

Para abrir una imagen desde IRAF se utiliza el comando `display`. El comando `display` abrirá la imagen especificada como argumento en la línea de comandos, o mediante el conjunto de valores de sus parámetros, con las opciones que se le entreguen.

Por ejemplo:

```
cl> display dev$pix 1
```

Esto abrirá la imagen de M51 empacada por defecto con IRAF en el frame 1.

`ds9` permite el uso de más de un frame, hasta 16, simultáneamente. En un segundo frame se puede abrir la misma imagen pero con su intensidad mapeada en una escala logarítmica:

```
cl>display dev$pix 2 ztrans=log
```

La imagen anterior desaparece y ahora hay una presentación diferente de los mismos datos. Si se presiona la tecla `Tab` teniendo activa la ventana de `ds9`, se puede saltar de un frame al otro.

12. Copiado y eliminación de imágenes

IRAF tiene sus propios comandos de copiado y eliminación de imágenes: `imcopy` e `imdelete`.

En particular, `imcopy` puede resultar más conveniente porque permite copiar segmentos de imagen.

Por ejemplo, para copiar la región localizada entre las columnas 180 y 320 y las filas 200 y 310, y guardar el resultado en el archivo de imagen `segment.fits`, se puede usar:

```
cl>imcopy dev$pix[180:320,200:310] segment.fits
cl>display segment.fits 3
```

13. Estadísticas y examen de imágenes

13.1. `imstatistics`

El task `imstatistics` produce estadísticos básicos de una imagen o sección de imagen. Por ejemplo:

```
cl>imstat dev$pix fields="image,mean,midpt,mode,stddev"
```

Esto calculará la media, mediana, punto medio, moda y desviación estándar para la distribución de píxeles en toda la imagen `dev$pix`, imprimiendo el resultado en pantalla con cinco columnas, cada una correspondiente a los campos indicados.

13.2. `imexamine`

El task `imexamine` permite examinar una imagen en interacción con el frame indicado. En el documento se muestra su ejecución como:

```
cl>imexamine segments.fits 2
```

14. Otros comandos básicos para operar sobre imágenes

14.1. `imarith`

El task `imarith` ejecuta operaciones aritméticas entre imágenes. Funciona según la sintaxis:

```
cl>imarith imagen1 operacion imagen2 resultado
```

donde `imagen1` e `imagen2` son los operandos, es decir, dos archivos de imagen pre-existentes, mientras que `resultado` será un archivo de imagen nuevo que se creará como resultado de la operación.

Las operaciones que se pueden realizar con `imarith` son:

- `+`: suma.

- -: resta.
- *: multiplicación.
- /: división.
- min: operación mínimo.
- max: operación máximo.

14.2. `imcombine`

El task `imcombine` combina un conjunto de imágenes a través de una operación especificada: promedio, mediana, sumas pesadas, cuadraturas y cuadraturas pesadas.

Este task es extremadamente útil durante los procesos de reducción, para hacer, por ejemplo, la combinación de archivos de calibración como superdarks o superbias.

14.3. `imshift`

Si se necesita desplazar una imagen, por ejemplo para alinear un conjunto de imágenes tomadas en diferentes bandas y hacer una combinación de colores, el task indicado es `imshift`.

`imshift` hace desplazamientos horizontales y verticales en una imagen un conjunto dado de píxeles. Por ejemplo:

```
cl>imshift imagenI imagen0 5 5
```

Esto inducirá un desplazamiento de 5 píxeles a la derecha y 5 píxeles hacia arriba en la imagen `imagenI`, y producirá como resultado una nueva imagen `imagen0` con las nuevas posiciones desplazadas.

14.4. `imtranspose`

Este task hace una transposición de la o las imágenes de entrada. Recordando que una imagen bidimensional se puede ver como una matriz, `imtranspose` intercambia filas por columnas en la imagen de entrada y como resultado produce una nueva imagen con la transposición.

Su sintaxis es:

```
cl>imtranspose imagenI imagen0
```

14.5. `rotate`

El task `rotate` es una versión más completa que `imshift`, en el sentido de que induce rotaciones y traslaciones arbitrarias sobre una imagen.

15. Graficación con `implot`

Otro task importante, y que puede ser usado en modo interactivo, es `implot`. La función básica de `implot` es graficar líneas y columnas de una imagen.

Si se quiere graficar toda la columna 50, es decir, la coordenada x igual a 50, de la imagen expuesta en el visor `ds9`, se puede usar:

```
cl>implot dev$pix[50,*]
```

Nuevamente se usa la sintaxis de bloque de imagen. Así como con `imcopy`, en general IRAF puede trabajar con secciones de imagen delimitadas por la sintaxis:

```
[xmin:xmax, ymin:ymax]
```

Indicando el rango en y con `*`, se liberan todos los posibles valores de y en la gráfica.

La ejecución de `implot` quita el control al prompt de IRAF y se lo entrega al cursor sobre la ventana de `ximtool`. Las opciones básicas en esta consola interactiva son:

```
:c #          grafica la columna #
:l #          grafica la linea #
q            finaliza la ejecución del modo interactivo
:o          mantiene la grafica (overplot)
:i<filename> nuevo archivo para graficar
```